DES3UE WS 2026 Übung 5

Abgabetermin: siehe E-Learning, Abgabeform elektronisch

☒ **DES3UEG1 (Niklas)** **Name** Klemens Danner **Aufwand in h** 7,5
☐ **DES3UEG2 (Niklas)** **Punkte** _____ **Kurzzeichen Tutor*in** _____
☐ **DES3UEG3 (Teufl)**

Geben Sie in Ihrer Ausarbeitung immer das gesamte Ergebnis der Abfrage aus (gilt auch für Beispiele mit Ergebnis-Auszug in der Angabe).

1. Data Dictionary **(5 Punkte** – 1 + 1,5 + 1 + 1,5**)**

Das Data Dictionary von Oracle speichert alle Informationen, die zur Verwaltung der Objekte (z.B. Tabellen, Sichten, Indizes, Prozeduren, Funktionen, Trigger) in der Datenbank benötigt werden. Obwohl dies üblicherweise in den Zuständigkeitsbereich des Datenbankadministrators fällt, stellt das Data Dictionary auch für Entwickler und Datenbankanwender eine wertvolle Informationsquelle dar. Sie lernen in den folgenden Aufgaben ausgewählte Bereiche des Data Dictionary aus der Perspektive des Datenbankanwenders kennen.

a. Erstellen Sie ein Skript für eine angegebene Tabelle, das die Spaltennamen, die Datentypen und die Länge der Datentypen sowie eine Information darüber liefert, ob Nullwerte zulässig sind. Fordern Sie den*die Benutzer*in auf, den Tabellennamen einzugeben (&-Operator). Geben Sie auch die Spalten `DATA_PRECISION` und `DATA_SCALE` aus und weisen Sie ihnen geeignete Aliasnamen zu. Testen Sie das Skript mit einer geeigneten Tabelle (zB `employees`).

b. Erstellen Sie ein Skript, das den Namen des Index und der dafür definierten Spalte(n) (in der richtigen Reihenfolge) ausgibt. Schließen Sie nur Indizes ein, die die Bezeichnung „`FILM`“ im Namen tragen.

Ergebnis-Auszug:

INDEX_NAME	COLUMNS
FILM_ACTOR_PK	ACTOR_ID, FILM_ID

c. Verwenden Sie die Tabelle `user_segments` und geben Sie die Namen und Größe der drei Tabellen aus, die den meisten Speicherplatz verwenden.

d. Bereiten Sie ein Skript vor, das Ihnen am Ende des Semesters ermöglicht, alle ihre angelegten Objekte zu entfernen. Erstellen Sie dafür eine Abfrage auf die Tabelle `user_objects` und generieren Sie die `DROP`-Statements entsprechend.
Achtung: Führen Sie die generierten Statements nicht aus, ansonsten entfernen Sie auch die noch benötigten Tabellen des HR- und Sakila-Schemas!

2. Materialisierte Sichten (Sakila-Datenbank) (6,5 Punkte – 2 + 1,5 + 1,5 + 1 + 0,5)

a. Formulieren Sie eine Anfrage, die das Alter der Schauspieler:innen zum aktuellen Tag berechnet. Geben Sie die ID, Vor- und Nachname und die Anzahl der abgeschlossenen Lebensjahre aus. Erstellen Sie daraus eine **virtuelle Sicht** `actor_age`. Fragen Sie die erstellte Sicht ab.

Ergebnis-Auszug:

	ACTOR_ID	FIRST_NAME	LAST_NAME	AGE
1	1	PENELOPE	GUINESS	75
2	2	NICK	WAHLBERG	66
3	3	ED	CHASE	56
4	4	JENNIFER	DAVIS	74

b. Erzeugen Sie aus der virtuellen Sicht `actor_age` eine materialisierte Sicht `actor_age_daily` mit kompletter Neuberechnung (Re-Materialisierung), die täglich um 01:00 (nachts) neu berechnet wird.

c. Verändern Sie den Aktualisierungszeitpunkt der erstellten materialisierten Sicht `actor_age_daily` in der Weise, dass sie jeweils nur mehr in der Nacht von Sonntag auf Montag um 03:00 Uhr aktualisiert wird. Speichern Sie die Sicht unter dem Namen `actor_age_weekly`.

d. Warten Sie im besten Fall (je nach Zeitpunkt der Ausarbeitung Ihrer Übung) die nächste Aktualisierung der materialisierten Sicht ab. Prüfen Sie jedenfalls im Data Dictionary den Zeitpunkt der letzten und der nächsten Aktualisierung. Um die richtigen Ansichten im DataDictionary zu finden, verwenden Sie das Stichwort „refresh“.

e. Löschen Sie die virtuelle und materialisierten Sichten anschließend wieder.

3. Hierarchische Abfragen – Human Resources (5 Punkte – 1,5 + 1,5 + 2)

a. Erstellen Sie eine hierarchische Abfrage, um die Angestelltennummer, die Managernummer und den Nachnamen für alle Angestellte unter De Haan anzuzeigen, die sich genau zwei Ebenen unterhalb dieses Angestellten (De Haan, Angestelltennummer: 102) befinden. Zeigen Sie zudem die Ebene dieser Angestellten an. *(2 Zeilen)*

b. Erstellen Sie eine Abfrage, um die gesamte Angestellten-Hierarchie (d.h. ausgehend von ‚`King`‘) zu durchlaufen. Zeigen Sie Untergebene nur an, wenn deren Vorgesetzte*r in einer anderen Abteilung arbeitet. Tipp: Sie können das Schlüsselwort PRIOR auch in der WHERE-Clause verwenden. *(6 Zeilen, ‚Higgins‘ befindet sich in der Ergebnis-Menge)*

c. Erstellen Sie eine Abfrage und ermitteln Sie, wie viele Untergebene es pro Manager gibt, deren Einstellungsdatum vor dem Manager liegen. Berücksichtigen Sie jeweils die gesamte Hierarchie dh. nicht nur direkte Untergebene bzw. Vorgesetzte. Geben Sie den Nachnamen und die Anzahl aus, Tipp: Sie können den `CONNECT_BY_ROOT` Operator verwenden. *(4 Zeilen, ‚Kochhar‘ hat einen Untergebenen, der davor eingestellt wurde)*

4. Hierarchische Abfragen – Sakila-Datenbank (7,5 Punkte)

HINWEIS:

- Verwenden Sie `EXECUTE SYS.KILL_MY_SESSIONS()` um hängen gebliebene Sessions/Abfragen zu beenden.

Die Schauspieler*innen Judy Dean und Russell Close haben sich 1990 bei Dreharbeiten kennengelernt und dabei beschlossen, die Aging Actors Association (Triple A) zu gründen. Noch im selben Jahr haben sie allen anderen Kolleg*innen davon erzählt, die sie bei gemeinsamen Dreharbeiten getroffen haben und die haben es erneut weitererzählt und so weiter, und so fort.

Sie sollen nun herausfinden, wie viele weitere Personen in diesem Jahr von der Vereinigung bereits gehört haben und damit mögliche Interessent*innen sein könnten. Zählen Sie direkte Weitererzählung der zwei Gründungsmitglieder und auch indirekte über weitere Personen. Jeder gemeinsame Film-Dreh bietet Gelegenheit für eine Kommunikation. Da die genauen Drehdaten nicht bekannt sind, werden zur Vereinfachung nur Filme berücksichtigt, die 1990 erschienen sind. Die zeitliche Reihenfolge der Drehs innerhalb dieses Jahres spielt keine Rolle. Gehen Sie folgendermaßen vor:

a) Erstellen Sie zuerst eine View `partners`, die Ihnen zu allen Filmen, die im Jahr 1990 erschienen sind, die Personen auflisten, die miteinander gedreht haben. Die View soll die `film_id` und die beiden `actor_ids` enthalten. Beispiel: Haben A, B und C gemeinsam in einem Film gespielt, enthält die View für diesen Film die Zeilen (A, B), (A, C), (B, A), (B, C), (C, A) und (C, B), sh. auch Beispiel-Ausgabe. Überprüfen Sie das Ergebnis, indem Sie die Anzahl der gefundenen Zeilen in der View ausgeben. *(3 Punkte, 922 Zeilen)*

Beispiel-Ausgabe der View:

	FILM_ID	ACTOR_ID1	ACTOR_ID2
1	860	126	123
2	860	126	64
3	860	123	126
4	860	123	64
5	860	64	126
6	860	64	123

b) Erstellen Sie aufbauend auf der obigen View eine Liste an möglichen Schauspieler*innen, die von der Triple A gehört haben könnten. Starten Sie bei den beiden Schauspielern die Rekursion und durchlaufen Sie diese **nicht** innerhalb eines Films. Gehen Sie davon aus, dass maximal bei drei Dreharbeiten von der Idee (weiter-)erzählt wurde (Rekursionstiefe). Geben Sie Vor- und Nachnamen aller Schauspieler*innen aus, die davon gehört haben könnten. *(3 Punkte, 48 Zeilen)* nicht innerhalb eines Films, zweite Bedingung nötig. bei 3 stops aufhören

c) Wie oft müsste die Info mindestens weitergegeben werden (= Anzahl der Weitererzählungen), damit alle (indirekt von den Gründungsmitgliedern erreichbaren) Schauspieler*innen, die in einem Film, der 1990 erschienen ist, mitgespielt haben von Triple A erfahren haben? Sie können zur Lösung der Aufgabe auf die Abfrage aus b) aufbauen. *(1,5 Punkte)*

Aufgabe 1

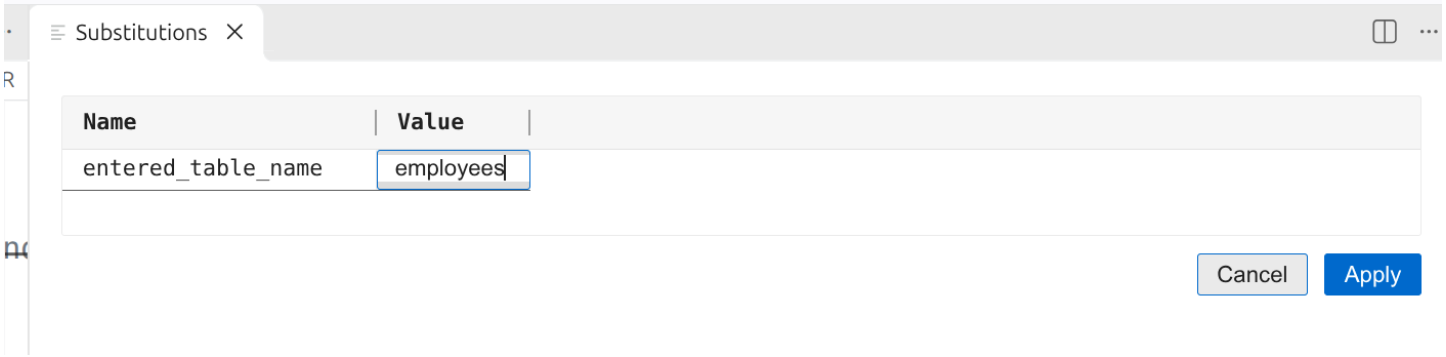
1a)

```
--a)
SELECT *
FROM dictionary
WHERE table_name LIKE 'USER_T%';

-- USER_TAB_COLUMNS sieht gut aus: Columns of user's tables, views and clusters
-- mal reinschauen

SELECT * FROM USER_TAB_COLUMNS;
-- sieht gut aus! alle gewollten Infos stehen darin.


SELECT column_name,
    data_type,
    data_type,
    data_length,
    nullable,
    data_precision AS ct_digits,
    data_scale AS ct_decimal_digits
FROM USER_TAB_COLUMNS
WHERE table_name = UPPER('&entered_table_name');
```



Ausgabe bei Eingabe von employees

	COLUMN_NAME	DATA_TYPE	DATA_TYPE	DATA_LENGTH	NULLABLE	CT_DIGITS	CT_DECIMAL_DIGITS
1	EMPLOYEE_ID	NUMBER	NUMBER	22	N	6	0
2	FIRST_NAME	VARCHAR2	VARCHAR2	20	Y	(null)	(null)
3	LAST_NAME	VARCHAR2	VARCHAR2	25	N	(null)	(null)
4	EMAIL	VARCHAR2	VARCHAR2	25	N	(null)	(null)
5	PHONE_NUMBER	VARCHAR2	VARCHAR2	20	Y	(null)	(null)
6	HIRE_DATE	DATE	DATE	7	N	(null)	(null)
7	JOB_ID	VARCHAR2	VARCHAR2	10	N	(null)	(null)
8	SALARY	NUMBER	NUMBER	22	Y	8	2
9	COMMISSION_PCT	NUMBER	NUMBER	22	Y	2	2
10	MANAGER_ID	NUMBER	NUMBER	22	Y	6	0
11	DEPARTMENT_ID	NUMBER	NUMBER	22	Y	4	0

1b)

Code

```
SELECT * FROM USER_TAB_COLUMNS WHERE table_name = 'EMPLOYEES';
SELECT * FROM USER_IND_columns WHERE table_name = 'FILM_ACTOR';
SELECT * FROM USER_IND_columns WHERE index_name = 'FILM_ACTOR_PK';


SELECT index_name,
LISTAGG(column_name, ', ')
WITHIN GROUP (ORDER BY column_position) AS columns
FROM USER_IND_COLUMNS
WHERE index_name LIKE '%FILM%'
GROUP BY index_name;
```

Ausgabe

	INDEX_NAME	COLUMNS
1	FILM_ACTOR_PK	ACTOR_ID, FILM_ID
2	FILM_CATEGORY_PK	FILM_ID, CATEGORY_ID
3	FILM_PK	FILM_ID

3 selected

1c)

Code

```
WITH tmp_table AS (
SELECT segment_name AS name,
       bytes AS size_bytes,
       DENSE_RANK() OVER (
            ORDER BY bytes DESC
       ) AS dense_rank
FROM user_segments
WHERE segment_type = 'TABLE'
)

SELECT name, size_bytes
FROM tmp_table
WHERE dense_rank <= 3;
```

Ausgabe

	NAME	SIZE_BYTES
1	RENTAL	917504
2	PAYMENT	851968
3	FILM	262144

3 selected

1d)

Code

```
SELECT * FROM USER_OBJECTS;

SELECT DISTINCT object_type
FROM user_objects
ORDER BY object_type;

SELECT CASE WHEN object_type = 'TABLE' THEN
              'DROP TABLE' || ' ' || object_name || ' CASCADE CONSTRAINTS' || ';'
            ELSE
              'DROP ' || object_type || ' ' || object_name || ';'
            END AS drop_statement
FROM user_objects
WHERE object_type IN (
      'SEQUENCE',
      'TABLE',
      'VIEW',
      'MATERIALIZED VIEW'
);
```

Ausgabe

	DROP_STATEMENT
1	DROP TABLE ACTOR CASCADE CONSTRAINTS;
2	DROP VIEW ACTOR_AGE;
3	DROP TABLE ACTOR_AGE_DAILY CASCADE CONSTRAINTS;
4	DROP MATERIALIZED VIEW ACTOR_AGE_DAILY;
...	
33	DROP TABLE STORE CASCADE CONSTRAINTS;

33 selected

Aufgabe 2

2a

Code

```
--a alter der Schauspieler zum aktuellen Tag,
-- virtuelle Sicht dazu erstellen und abfragen

CREATE OR REPLACE VIEW actor_age AS
SELECT actor_id, first_name, last_name,
    TRUNC(MONTHS_BETWEEN(SYSDATE, birth_date) / 12) AS age
FROM actor
ORDER BY actor_id;

SELECT *
FROM actor_age;
```

Ausgabe

	ACTOR_ID	FIRST_NAME	LAST_NAME	AGE
1	1	PENELOPE	GUINESS	75
2	2	NICK	WAHLBERG	66
3	3	ED	CHASE	56
4	4	JENNIFER	DAVIS	74
5	5	JOHNNY	LOLLOBRIGIDA	79
6	6	BETTE	NICHOLSON	71

...

200	200	THORA	TEMPLE	65

200 selected

2b

Code

```
CREATE MATERIALIZED VIEW actor_age_daily
    REFRESH COMPLETE
    START WITH TRUNC(SYSDATE + 1) + 1/24
    NEXT TRUNC(SYSDATE + 1) + 1/24
AS
SELECT *
FROM actor_age;

-- abfragen, für die doku
SELECT * FROM actor_age_daily;
```

Ausgabe

```
-- selbe Ausgabe wie in 2a
```

2c

Code

```
--c nur in der Nacht von Sonntag auf montag 3.00 uhr
CREATE MATERIALIZED VIEW actor_age_weekly
    REFRESH COMPLETE
    START WITH TRUNC((SYSDATE + 7), 'IW') + 3/24
    NEXT TRUNC((SYSDATE + 7), 'IW') + 3/24
AS
SELECT *
FROM actor_age;

SELECT * FROM actor_age_weekly;
```

Ausgabe

```
-- selbe Ausgabe wie in 2a
```

2d)

Ich suche den Zeitpunkt der letzten und nächsten Aktualisierung im Data Dictionary.
Also:

```
-- nach dem Stichwort refresh
SELECT *
FROM dictionary
WHERE table_name LIKE '%refresh%'
OR comments LIKE '%refresh%';
```

	TABLE_NAME	COMMENTS
1	USER_REFRESH	All the refresh groups
2	USER_REFRESH_CHILDREN	All the objects in refresh groups, where the user owns the refresh group
3	USER_MVIEW_REFRESH_TIMES	Materialized views and their last refresh times for each master table that the user can look at
4	ALL_MVIEW_REFRESH_TIMES	Materialized views and their last refresh times for each master table that the user can look at
5	ALL_REFRESH	All the refresh groups that the user can touch
6	ALL_REFRESH_CHILDREN	All the objects in refresh groups, where the user can touch the group
7	ALL_REFRESH_DEPENDENCIES	Description of the detail tables that materialized views depend on for refresh

Ich suche also weiter in: USER_MVIEW_REFRESH_TIMES

```
SELECT *
FROM USER_MVIEW_REFRESH_TIMES;
```

	OWNER	NAME	MASTER_OWNER	MASTER	LAST_REFRESH
1	S2510307051	ACTOR_AGE_DAILY	S2510307051	ACTOR	10-NOV-26
2	S2510307051	ACTOR_AGE_WEEKLY	S2510307051	ACTOR	10-NOV-26

LAST_REFRESH habe ich also gefunden - heute, der 10. November wurde die Sicht erstellt, daher der letzte refresh auch heute.
Weitersuchen nach dem next refresh

Ich probiere USER_REFRESH:

```
SELECT *
FROM USER_REFRESH;
```

	ROWNER	RNAME	REFGROUP	IMPLICIT_DESTROY	PUSH_DEFERRED_RPC	REFRESH_AFTER_ERRORS	ROLLBACK_SEG	JOB	NEXT_DATE	INTERVAL
1	S2510307051	ACTOR_AGE_DAILY	2753	Y	Y	N	(null)	0	11-NOV-26	TRUNC(SYSDATE + 1) + 1/24
2	S2510307051	ACTOR_AGE_WEEKLY	2756	Y	Y	N	(null)	0	16-NOV-26	TRUNC((SYSDATE + 7), 'IW') + 3/24

Top, hier steht das next_date. Nochmal schöner:

```
SELECT rname, next_date
FROM USER_REFRESH;
```

	RNAME	NEXT_DATE
1	ACTOR_AGE_DAILY	11-NOV-26
2	ACTOR_AGE_WEEKLY	16-NOV-26

Interessanterweise findet man den last_refresh und das next_date in zwei unterschiedlichen Tabellen.
Wir haben aber beide gefunden.

2e)

Code

```
DROP MATERIALIZED VIEW actor_age_weekly;
DROP MATERIALIZED VIEW actor_age_daily;
DROP VIEW actor_age;
```

Ausgabe

```
Materialized view ACTOR_AGE_WEEKLY dropped.


Materialized view ACTOR_AGE_DAILY dropped.


View ACTOR_AGE dropped.
```

Aufgabe 3

3a)

Code

```
-- alle employees die genau 2 levels unter 102 sind
-- level 1 ist 102
-- zwei darunter ist level 3
SELECT employee_id, last_name, manager_id, LEVEL
FROM employees
WHERE LEVEL = 3
START WITH employee_id = 102
CONNECT BY PRIOR employee_id = manager_id
      AND LEVEL <= 3; -- Rekursion begrenzen
-- heißt: geh weiter wo die manager_id 102 ist (beim ersten Schritt)
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME	MANAGER_ID	LEVEL
1	104	Ernst	103	3
2	107	Lorentz	103	3

2 selected

3b)

Code

```
SELECT employee_id, last_name
FROM employees
WHERE last_name = 'King';
-- King -- 100

SELECT employee_id, last_name, manager_id, department_id
FROM employees
WHERE department_id != PRIOR department_id
START WITH employee_id = 100
CONNECT BY NOCYCLE PRIOR employee_id = manager_id;
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME	MANAGER_ID	DEPARTMENT_ID
1	200	Whalen	101	10
2	205	Higgins	101	110
3	103	Hunold	102	60
4	124	Mourgos	100	50
5	149	Zlotkey	100	80
6	201	Hartstein	100	20

6 selected

3c)

Code

```
WITH tmp_table AS (
      SELECT CONNECT_BY_ROOT employee_id AS root_manager_id,
            -- jede id wird rek. durchlaufen
            -- erste id ist also manager aller anderen
            CONNECT_BY_ROOT last_name AS root_manager_name,
            employee_id, last_name,
            manager_id, hire_date
      FROM employees
      WHERE hire_date < CONNECT_BY_ROOT hire_date
      -- hire_date von Untergebenem war vorher
      CONNECT BY NOCYCLE PRIOR employee_id = manager_id
)

SELECT root_manager_id, root_manager_name,
      COUNT(*) AS untergebene
FROM tmp_table
GROUP BY root_manager_id, root_manager_name;
```

Ausgabe

	ROOT_MANAGER_ID	ROOT_MANAGER_NAME	UNTERGEBENE
1	101	Kochhar	1
2	124	Mourgos	4
3	149	Zlotkey	3
4	102	De Haan	2

4 selected

Aufgabe 4

4a)

Code

```
CREATE OR REPLACE VIEW partners AS
SELECT film.film_id AS film_id,
       id1.actor_id AS actor_id1,
       id2.actor_id AS actor_id2
FROM film_actor id1
       JOIN film_actor id2
              ON id1.film_id = id2.film_id
       JOIN film
              ON id1.film_id = film.film_id
WHERE release_year = 1990
       AND id1.actor_id <> id2.actor_id;

SELECT *
FROM partners;
```

Ausgabe

	FILM_ID	ACTOR_ID1	ACTOR_ID2
1	597	11	28
2	597	13	28
3	597	64	28
4	597	129	28
5	597	184	28
...			
921	714	166	182
922	714	200	182

922 selected

4b)

Code

```
-- actor_ids der beiden:
-- Judy Dean: 35
SELECT actor_id, last_name, first_name
FROM actor
WHERE last_name = 'DEAN' AND first_name = 'JUDY';

-- Russell Close: 183
SELECT actor_id, last_name, first_name
FROM actor
WHERE last_name = 'CLOSE' AND first_name = 'RUSSELL';

-- von den beiden die view durchlaufen,
-- nur bis level 3

WITH partners_reached AS (
   SELECT DISTINCT actor_id2
   FROM partners
   START WITH actor_id1 IN (183, 35)
   CONNECT BY NOCYCLE PRIOR actor_id2 = actor_id1
   AND LEVEL <= 3
   AND PRIOR film_id <> film_id
)
SELECT pre.actor_id2, act.first_name, act.last_name
FROM partners_reached pre
JOIN actor act ON pre.actor_id2 = act.actor_id;
```

Ausgabe

	ACTOR_ID2	FIRST_NAME	LAST_NAME
1	194	MERYL	ALLEN
2	3	ED	CHASE
3	10	CHRISTIAN	GABLE
4	15	CUBA	OLIVIER
5	20	LUCILLE	TRACY
6	21	KIRSTEN	PALTROW
7	31	SISSY	SOBIESKI

...

48	186	JULIA	ZELLWEGER

48 selected